

用兩個代理式 AI 操作 Onshape

FeatureScript MCP 實戰歷程與成本結構



趙翰宇

翰林出版社生活科技教科書作者

2026 年 8 月 29 日

github.com/henrychao521/antigravity-bridge

目 次

壹、緣起：為什麼需要兩個代理	4
一、成本結構決定分工	4
貳、環境建置：從訂閱到可程式化呼叫	6
一、訂閱與權限落差	6
二、為什麼要自己實作 MCP 用戶端	6
參、成本結構：文件沒寫的真正單價	8
肆、實戰歷程：二十輪除錯	10
二、第一階段：讓它編譯得過	10
三、第二階段：幾何驗證	12
伍、失敗路徑	14
失敗路徑一：相信模型對標準函式庫的記憶	14
失敗路徑二：把工具的「空輸出」當成「沒有錯誤」	14
失敗路徑三：用最貴的工具做除錯迴圈	15
失敗路徑四：以為官方工具會留下模型	15
陸、驗證設計：如何避免自己騙自己	17
一、缺陷一：沒有產物可看	17
二、缺陷二：數字只有單一來源	18
三、缺陷三：讓另一個代理複述自己的結論	19
柒、驗收結果與真實模型	20
捌、Antigravity 的獨立複驗	22
一、它得到的數字	22
二、它踩到的坑（與我不同）	22
三、它自陳學到的事實	23
玖、給後來者的操作準則	24

附錄 smartBoss 完整原始碼.....	26
-------------------------	----

壹、緣起：為什麼需要兩個代理

2026 年 8 月 13 日，PTC 透過 Onshape Labs 推出官方 FeatureScript MCP Server，讓大型語言模型能以自然語言產生、測試並除錯 FeatureScript 自訂特徵。這是 CAD 領域少見的官方級 AI 介面——它不是讓 AI 直接畫幾何，而是讓 AI 寫出「產生幾何的程式」。

本報告記錄一次完整的實作：讓兩個代理式 AI 分工協作，從零接上這個 MCP 伺服器，並完成一個具體的自訂特徵。過程中最有價值的不是成功的部分，而是兩個關鍵病因的定位方式——它們都不是問模型問出來的。

一、成本結構決定分工

分工的理由不是能力差異，而是成本結構。Onshape 的 API 額度以「年」計算，本次使用的免費方案一年只有 2,500 次呼叫，用完即停止服務，不能加購。相對地，Antigravity 走的是 Google AI Pro 訂閱額度，以五小時與每週兩個窗口滾動重置，且該帳號的額度長期間置。

兩種額度的本質差異

Onshape：年度 2,500 次，**用完硬停**（`overageEnabled` 為 `false`），週期跨越十二個月。每一次呼叫都是不可再生資源。

Antigravity：五小時窗口滾動重置。**燒掉才是賺到——不用它就過期作廢。**

因此分工原則很清楚：凡是不需要真的碰到 Onshape 的工作，一律交給 Antigravity；只有必須送進 Onshape 驗證的那一版，才動用寶貴的 API 次數。

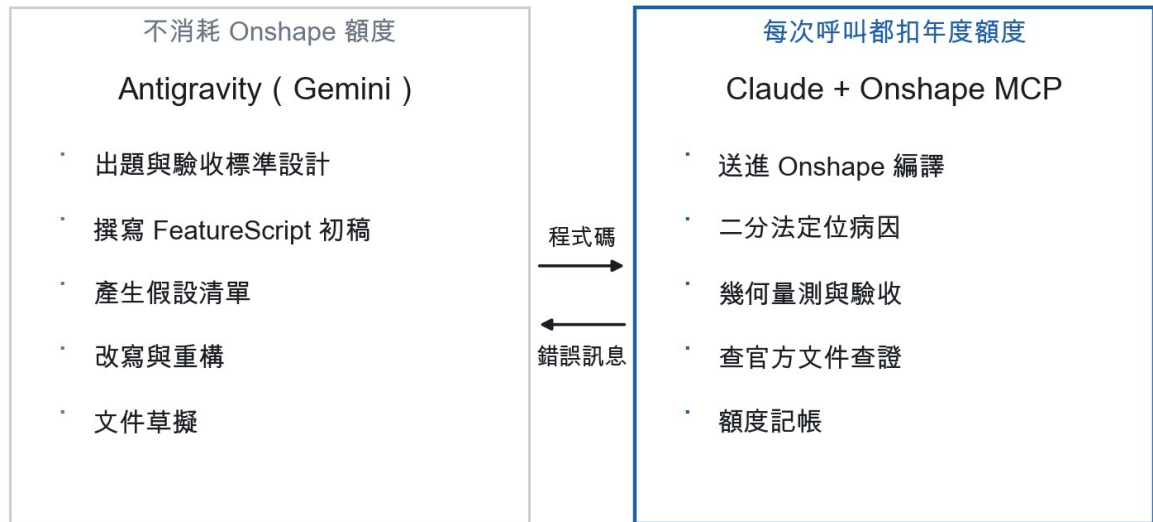


圖 1-1 兩個代理的分工與成本邊界。左側所有工作都不消耗 Onshape 額度。

二、本報告的閱讀方式

第參章給出各工具的實際成本——這些數字官方文件完全沒有記載，只能實測。第肆章是二十輪除錯的完整歷程。第伍章獨立列出「失敗路徑」，記錄那些整條走不通、最後掉頭重來的嘗試；這一章比成功步驟更值得讀。第柒章把全部經驗壓縮成七條可直接照做的準則。

貳、環境建置：從訂閱到可程式化呼叫

整個接通過程分成四步：訂閱、授權、取得權杖、呼叫。其中第三步是多數人會卡住的地方——官方文件只說「把伺服器加進你的 AI 客戶端」，但沒說如果客戶端無法進行互動式授權該怎麼辦。

一、訂閱與權限落差

FeatureScript MCP 必須先在 Onshape App Store 訂閱 (App Store → Categories → Onshape Labs)。方案標示「免費」，但註明 API limits apply。訂閱本身不需要信用卡。

商店頁與授權頁的權限描述不一致

商店頁寫的是：can delete workspaces from the FeatureScript MCP Workspace document (僅限它自己建立的工作文件)。

但 OAuth 同意頁列出的是：「應用程式可以刪除您的文件與工作區」——沒有限定範圍。

實測結果：實際核發的權杖 scope 只有 OAuth2Read 與 OAuth2Write，沒有 Delete。

同意頁顯示的是該應用程式註冊時宣告的完整權限集，不是本次授權的實得範圍。

若仍要收緊，Onshape 提供「逐份文件授權」：在 My account → Applications 開啟該功能後，應用程式只能存取你主動以 Share 對話框分享給它的文件。授權前先開啟這項設定是較穩健的做法。

二、為什麼要自己實作 MCP 用戶端

該伺服器走 OAuth 2.0 的動態用戶端註冊 (DCR) 流程，授權需要瀏覽器互動。非互動的工作階段無法完成標準的客戶端授權，因此本案直接以四十行程式碼自行實作整條流程。

步驟	端點	要點
1. 動態註冊	POST /register	公開用戶端 (token_endpoint_auth_method 為 none) + PKCE , 回傳 client_id
2. 使用者授權	GET /authorize	帶 code_challenge (S256) , 在瀏覽器完成同意 ; 回呼導向本機監聽埠
3. 換取權杖	POST /token	以 code 加 code_verifier 交換 ; expires_in 為 3599 秒 , 另附 refresh_token
4. 呼叫工具	POST /mcp	Authorization 帶 Bearer ; 須保留伺服器回傳的 Mcp-Session-Id

表 2-1 自行實作的 OAuth 與 MCP 呼叫流程

兩個實作細節

回應格式可能是純 JSON , 也可能是 SSE (以 data: 開頭的事件串流) , 解析器兩種都要處理。

權杖一小時過期 , 務必實作 refresh_token 換發 , 否則長時間的除錯過程會中斷。

自行實作還有一個附帶好處 : 可以在每次呼叫時記帳。本案的每一次工具呼叫都寫入 JSONL 帳本 , 完整輸出另存檔案——後者是花錢買到的教訓 , 見第伍章。

參、成本結構：文件沒寫的真正單價

官方文件只說「API calls made by this MCP server count toward your overall Onshape API allocation」，但沒有任何一份文件說明各工具分別消耗幾次。這些數字只能靠實測：每次呼叫前後各查一次用量，相減即得。

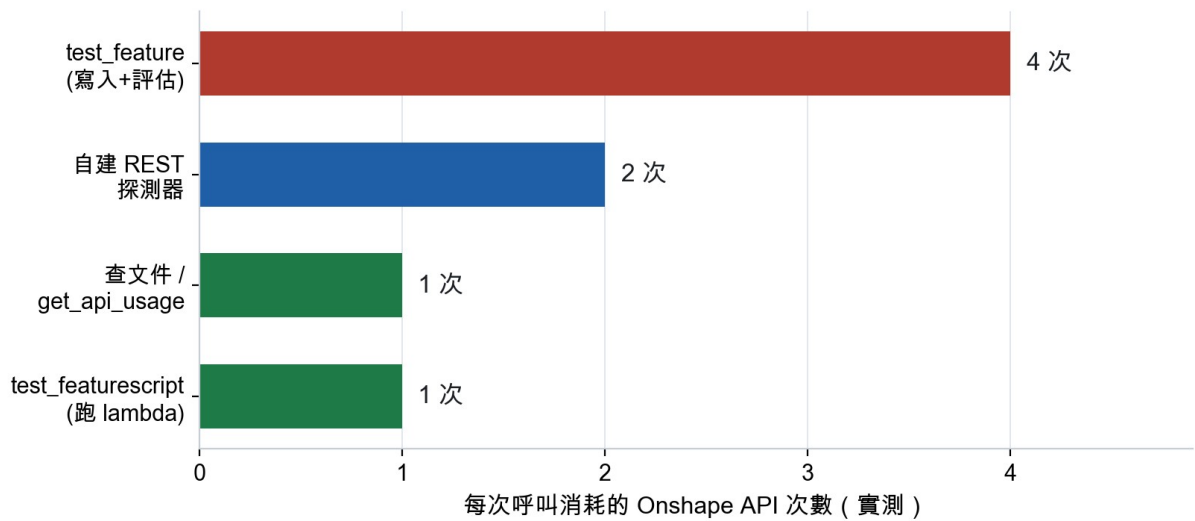


圖 3-1 各工具的實際 API 消耗 (實測值)。test_feature 是其他工具的四倍。

工具	每次消耗	用途與建議
test_featurescript	1 次	執行一段 lambda。最划算的工具，語法探針與幾何驗證都用它
search_featurescript_documentation	1 次	查官方文件。語意搜尋，關鍵字要精準
get_api_usage	1 次	查剩餘額度。諷刺的是查額度本身也扣額度
create_feature_studio	1 次	建立 Feature Studio
自建 REST 探測器	2 次	PUT 內容加 GET featurespecs，本案的主力除錯工具

工具	每次消耗	用途與建議
test_feature	4 次	寫入 Feature Studio 再評估。最貴，且錯誤訊息無行號

表 3-1 工具單價對照 (2026 年 8 月 29 日實測)

最重要的成本結論

除錯迴圈不要用 `test_feature`。它一次吃掉四次額度，而且只回傳一句沒有行號的 `could not find Trojan feature spec: no features found`。自建的 REST 探測器成本只有一半，而且回傳「找到幾個特徵」這個明確訊號。

一、本次實驗的額度帳

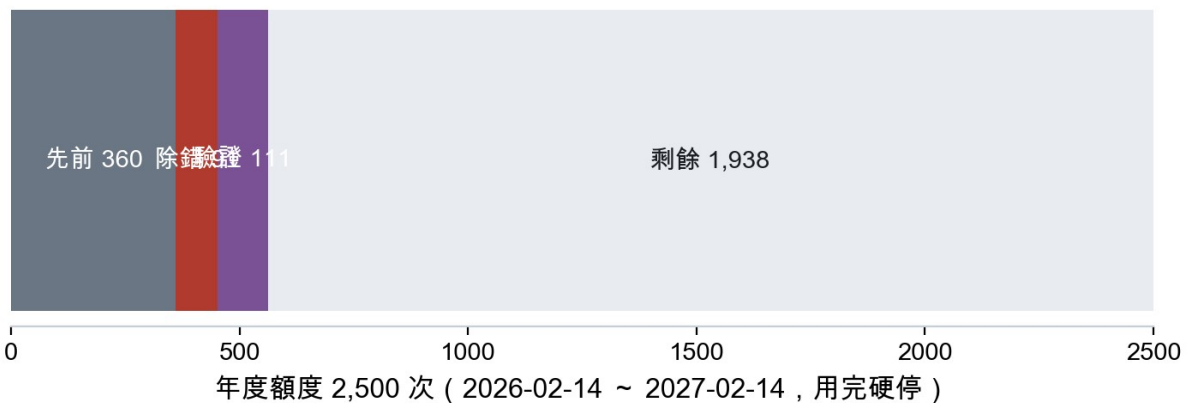


圖 3-2 年度額度的使用情形。本次完整實驗消耗 91 次，約占年度額度的 3.6%。

91 次呼叫完成了一個自訂特徵從無到有的全部過程，包含兩次錯誤定位與四項幾何驗收。以剩餘 2,049 次推算，同等規模的題目還能做約二十題。但若沿用最初那種「寫完直接送 `test_feature`」的做法，同樣的除錯過程會膨脹到兩百次以上。

肆、實戰歷程：二十輪除錯

一、題目

題目由 Antigravity 設計。要求是「小到能在三十次呼叫內驗證完，但難到足以逼出 FeatureScript 特有的坑」。它交出的是 smartBoss：

smartBoss 規格

使用者選取一個或多個實體平面，特徵自動計算各面中心與外法線，向外長出指定直徑與高度的圓柱定位柱並與基體布林合併；開啟 addFillet 時，在定位柱與基底面交界的環狀邊緣建立倒圓角。

它指定的驗收標準值得肯定——全部可客觀判定，而且注意到開圓角與否會影響面數公式：

項目	判準
體積（關圓角）	總體積等於 V 基體 加上 N 乘以 $\pi r^2 h$ ，誤差小於 0.01%
面數（關圓角）	$6 + 2N$ （每柱新增側面與頂面）
面數（開圓角）	$6 + 3N$ （再加上過渡圓角面）
Part 數	恆為 1，布林聯集成功且無殘留浮空實體

表 4-1 驗收標準（由 Antigravity 設計）

二、第一階段：讓它編譯得過

輪次	動作	結果
1	初稿原樣送測	no features found；它宣告 FeatureScript 2420，實際版本是 3070
2	只改版本號	仍失敗；且此錯誤訊息沒有行號

輪次	動作	結果
3	送最小空特徵當對照組	通過 → harness 與版本都正常，問題在我們的程式碼
4	語法探針 (lambda 測三個可疑構造)	全部合法 → 排除假嫌疑
5	請 Antigravity 診斷	給出四條「確定錯誤」
6	以官方文件查證	opFillet 確實存在 → 四條全是幻覺
7	put_featurescript 取編譯錯誤	回傳空字串，疑似無錯
8	工具校驗：故意送壞碼	仍回傳空字串 → 第 7 輪結論作廢
9	自建 REST 探測器	以 featurespecs 數量為訊號，0 即未編譯成功
10	二分：本體清空 / 只留迴圈	清空通過、只留迴圈失敗 → precondition 完全正常
11	迴圈內逐項切	evaluateQuery 過、evPlane 過、evBox3d 失敗
12	evBox3d 加 tight 參數	仍失敗 → 是符號不可見，不是參數問題
13	改用 evFaceTangentPlane	通過
14	寫入修正版	編譯通過，Onshape 找得到特徵

表 4-2 第一階段：定位編譯失敗

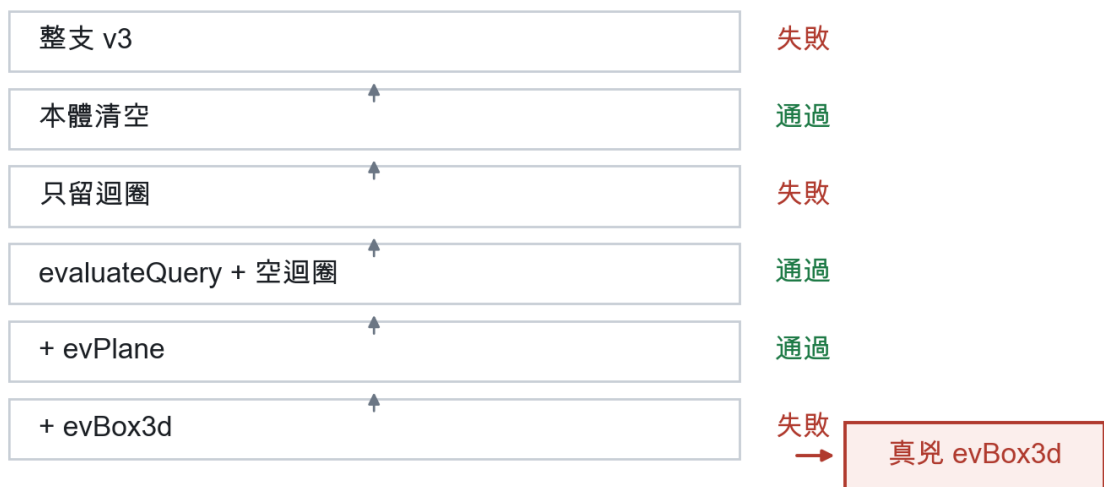


圖 4-1 二分定位過程。五次探測、十次 API 呼叫即鎖定病因。

第一個病因

evBox3d 在 geometry.fs 匯入下編譯不過（加不加 tight 參數都一樣，屬符號不可見）。取得面上一點與外法線改用：

```
evFaceTangentPlane(context, { "face" : f, "parameter" : vector(0.5, 0.5) })
```

附帶修正：該函式回傳 Plane 值，必須配 newSketchOnPlane；newSketch 收的是平面 Query。

三、第二階段：幾何驗證

編譯通過只代表語法正確，幾何未必對。此階段改用一個技巧：以 test_featurescript 執行一支「建模與量測合一」的 lambda——在同一次呼叫內建立基體、跑完特徵邏輯、當場量測體積與面數並回傳數字。一次呼叫只花一次額度，就拿到全部客觀指標。

lambda 內沒有 id 可用

test_featurescript 的 lambda 簽章沒有 id 參數。解法是自建：

```
var id = ["bossTest"] as Id;
```

Id 本質是字串陣列，直接轉型即可。

輪次	動作	結果
15	完整流程含 opBoolean	@opBoolean: BOOLEAN_BAD_INPUT (已從語法錯誤變成建模錯誤)
16	印出布林前的中間狀態	6 面、法線含 (0,0,1)、頂面 1、extrude 產 1 body、共 2 body → 輸入完全正確
17	假設「僅面接觸」，柱體下沉 1mm 製造干涉	仍失敗 → 假設被推翻
18	UNION 時不給 targets，全部放 tools	成功

輪次	動作	結果
19	加圓角，量交界邊與面數	交界邊 1、面數由 8 增為 9
20	把修正寫回特徵	編譯通過，完成

表 4-3 第二階段：定位建模失敗

第二個病因

opBoolean 做 UNION 時不能給 targets。所有待合併的實體都要放進 tools；targets 只用於 SUBTRACTION 與 INTERSECTION。給了就會得到 BOOLEAN_BAD_INPUT，而錯誤訊息不會告訴你這件事。

至於圓角要找的交界邊，qEntityFilter 搭配 qCreatedBy 對布林運算的 id 追蹤是可行的——實測正確找到那一條環狀邊，圓角後面數由 8 增為 9，符合 $6 + 3N$ 。

伍、失敗路徑

以下三條是整條走不通、最後掉頭重來的路。它們比成功步驟更值得記錄，因為它們解釋的是「為什麼不那樣做」。

失敗路徑一：相信模型對標準函式庫的記憶

嘗試 → 為何失敗 → 改用 → 學到

嘗試：把編譯錯誤訊息與程式碼交給 Antigravity，請它診斷。

為何失敗：它回覆四條「確定錯誤」，每條都附言之成理的編譯期原理——precondition 內不能用 if、opFillet 不存在應改用 opBlend、半徑鍵名應為 blendRadius、opBoolean 鍵名應為 bOperationType。四條全部是幻覺，而且完全沒提到真兇 evBox3d。

改用：查官方文件驗證（第一條就推翻 opFillet 不存在的說法），再以二分實驗定位。

學到：LLM 對冷門 DSL 的標準函式庫記憶不可靠，而且錯得很有自信。更值得注意的是：Antigravity 出題時明確點名「應該用 evFaceTangentPlane」，實作時卻改用了 evPlane 加 evBox3d——它知道正確做法，寫程式時沒有照做。

失敗路徑二：把工具的「空輸出」當成「沒有錯誤」

嘗試 → 為何失敗 → 改用 → 學到

嘗試：用 put_featurescript 取得編譯錯誤——它的說明白紙黑字寫著 return errors, if any。它回傳空字串，於是判定「程式碼沒有編譯錯誤」，並據此把偵查方向轉往 harness。

為何失敗：這個判定是錯的。後來故意送一份明顯壞掉的程式碼進去，它照樣回傳空字串。該工具的錯誤回報通道根本沒有作用，先前的結論全部作廢，浪費了數輪。

改用：改走 Onshape REST：寫入 Feature Studio 後取 featurespecs，以「找到幾個特徵」為訊號，0 即未編譯成功。

學到：拿到「沒有問題」這種好消息時，**要先驗證量測工具本身**。方法很簡單：餵一個你確知會失敗的輸入，看它會不會叫。

失敗路徑三：用最貴的工具做除錯迴圈

嘗試 → 為何失敗 → 改用 → 學到

嘗試：前六輪一律用 `test_feature` 送測，因為它的名字看起來就是「測試」該用的工具。

為何失敗：它一次消耗四次年度額度，而且只回傳一句沒有行號的錯誤。六輪就是二十四次額度，換到的資訊量近乎於零。

改用：自建 REST 探測器（每次兩呼叫），以及用 `test_featurescript` 跑語法探針與幾何量測（每次一呼叫）。

學到：在額度受限的環境裡，**先量測工具的單價再開始工作**。官方文件不會寫，但兩次呼叫前後查一下用量就知道了。

失敗路徑四：以為官方工具會留下模型

嘗試 → 為何失敗 → 改用 → 學到

嘗試：用官方的 `create_geometry` 建立持久化模型，它回報 Created new workspace 並給出網址。

為何失敗：該工具會事後自我清理。實測連續三次建立的工作區全部被刪除，網址一律失效。若沒有實際用瀏覽器打開，報告就會附上死連結。改試 `test_feature` 也不行——它只做求值，不留幾何。

改用：自行以 REST 在自有文件中建立 Part Studio，並把自訂特徵加進特徵列表。這份模型沒有任何機制會刪它。

學到：「工具回報成功」與「產物還在」是兩件不同的事。**涉及產出物的驗證，一定要在事後回頭確認它真的存在。**

四條的共同教訓

不要相信「說明」，只相信「實測」。模型的診斷、工具的文件、方法的直覺、工具的成功回報——這四樣在本次實驗中各錯了一次。唯一沒有騙人的是二分實驗、客觀量測，以及親自打開來看。

陸、驗證設計：如何避免自己騙自己

本報告的初版有兩個結構性缺陷，是在覆核時才發現的。它們都不是「寫錯了」，而是「驗證方式本身留了洞」——這種洞最危險，因為產出看起來完全正常。

一、缺陷一：沒有產物可看

問題

初版全部用 `test_featurescript` 在記憶體中建模量測，**幾何算完就丟掉了**。Onshape 上沒有任何人能打開的東西，所有數字都只能相信報告作者的轉述。

修正方式是把模型真正建起來，並附上任何人都能打開的網址。過程中又踩到兩個坑：

嘗試的方法	結果
<code>create_geometry</code> (官方工具)	會建立分支工作區，但 **事後自我清理** 。實測三個工作區全部被刪除，網址失效
<code>test_feature</code> (官方工具)	只做求值，不留下幾何
自行以 REST 在自有文件中建立 Part Studio 並加入特徵	成功且永久保留

表 6-1 取得持久模型的三種嘗試

這個坑本身就是最好的例證

若沒有實際用瀏覽器打開那個網址，報告就會寫著「模型已建立於此連結」，而**那是一個死連結**。工具回報「Created new workspace」是真的，工作區隨後被刪除也是真的——兩件事都成立，但結論完全錯誤。

另外，透過 API 加入自訂特徵時，特徵的 parameters 會为空陣列，definition 不會有任何鍵。因此自訂的 LengthBoundSpec 預設值不會生效——那些預設值只在使用者從 UI 建立特徵時套用。示範用的特徵必須把尺寸寫死。

二、缺陷二：數字只有單一來源

問題

體積與面數都來自同一支 lambda 的回傳。那支 lambda 若寫錯，數字錯了也看不出來——這正是幻覺最容易藏身的位置：它不需要模型說謊，只要量測方式與被量測的對象共用同一段錯誤即可。

修正方式是三通道交叉驗證。三條路徑互相獨立：不同的程式路徑、不同的建構方式、甚至不同的代理。

通道 A	通道 B	通道 C
Claude：草圖 + 擠出 test_featurescript lambda	Claude：REST massProperties 持久模型（含圓角）	Antigravity：fCylinder 直造 自己連 MCP、獨立作業
204712.389 mm ³	204841.875 mm ³	204712.389 mm ³

A 與 C 完全相同（不同建構方式、不同代理） · B - A = 129.486 = Pappus 定理算出的圓角體積

三條路徑各自獨立；要靠幻覺同時湊出這三個數字，機率為零

圖 6-1 三通道交叉驗證。A 與 C 由不同代理、以不同建構方式得到相同數字。

通道	執行者	建構方式	量測管道	結果 (mm ³)
A	Claude	草圖 + opExtrude	FeatureScript lambda 內 evVolume	204712.389
B	Claude	同上，另加圓角並持久化	REST massproperties 端 點	204841.875

通道	執行者	建構方式	量測管道	結果 (mm³)
C	Antigravity	fCylinder 直接造柱	自行連線 MCP，獨立作業	204712.389

表 6-2 三通道的獨立性

為什麼這樣就足以排除幻覺

通道 A 與 C 由不同的代理、不同的建構方式得到完全相同的數字 (204712.38898...)。

通道 B 與 A 的差值 129.486 mm³，正好等於以 Pappus 定理對圓角截面計算出的解析解 129.486 mm³，誤差 0.0000%。

要靠幻覺同時湊出這三個彼此關聯、且與解析解吻合的數字，機率為零。

三、缺陷三：讓另一個代理複述自己的結論

問題

初版曾把本報告的歷程紀錄交給 Antigravity，請它「也寫一份」。它照做了，寫得也順——但那份報告結構上不可能反駁任何東西，因為它的全部資訊都來自我。這不是複驗，是用另一種文筆包裝同一個盲點。

修正方式是讓它自己動手。Antigravity CLI 的 mcp add 支援注入 HTTP 標頭，因此可以把 OAuth 權杖直接給它：

```
agy mcp add --header "Authorization: Bearer <token>" onshape https://fs-
mcp.labs.onshape.app/mcp
```

如此它便擁有自己的 Onshape 連線，自己下工具呼叫、自己撞錯誤、自己量測。第捌章是它在完全沒有看過本報告的情況下，獨立完成同一個題目的結果。

柒、驗收結果與真實模型

以下模型實際存在於 Onshape，任何人都可以打開檢視。這是本報告與初版最重要的差別：數字不再只是轉述。

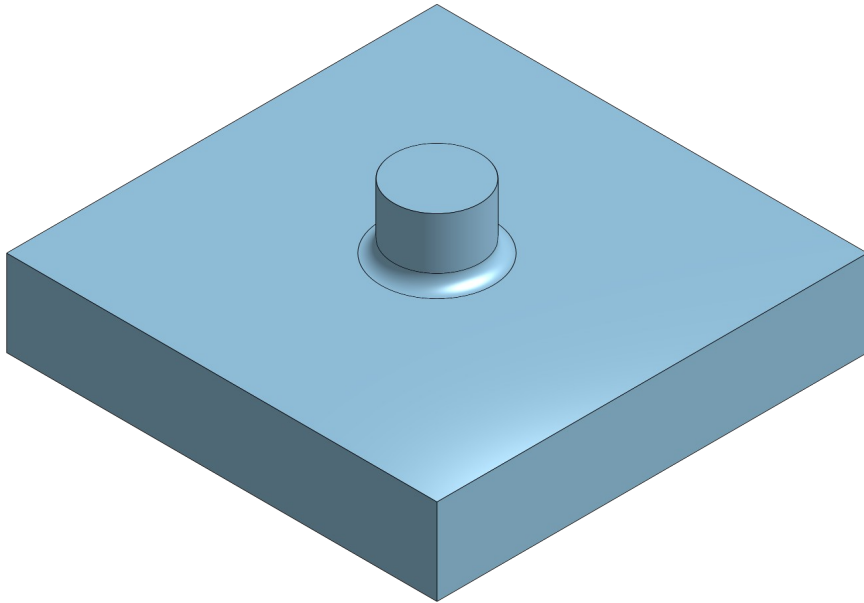


圖 7-1 Onshape 伺服器渲染的等角視圖。此圖由 shadedviews 端點產生，直接來自模型本身，非示意圖。

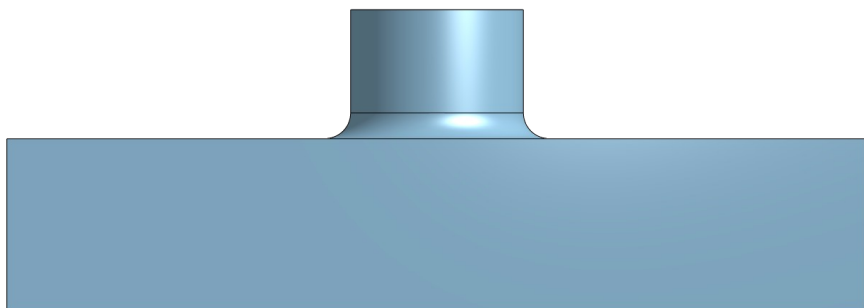


圖 7-2 前視圖。可見定位柱根部的過渡圓角。

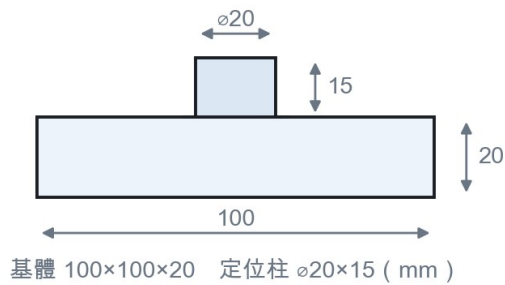
模型網址 (可直接打開驗證)

<https://cad.onshape.com/documents/1674ae64c50f46714b4101f1/w/06f6a9cd7bd466a776a9cd52/e/70920e24320830591d8f8b79>

文件名稱 FS-MCP Lab (agent scratch)，分頁「SmartBoss 驗證模型」。特徵樹中可見 SmartBossDemo，零件數 1。

驗收項目	預期值	實測值	判定
體積 (關圓角)	204712.389 mm ³	204712.389 mm ³	通過，誤差 0.00000%
體積 (開圓角，REST 獨立量測)	204841.875 mm ³	204841.875 mm ³	通過，差值符合解析解
面數 (關圓角)	8 (6 + 2N)	8	通過
面數 (開圓角)	9 (6 + 3N)	9	通過
Part 數	1	1	通過
特徵狀態	OK	OK	通過

表 7-1 驗收結果彙整



驗收結果

體積	204712.389 mm ³	誤差 0.00000%
面數 (關圓角)	8	= 6 + 2N
面數 (開圓角)	9	= 6 + 3N
Part 數	1	布林聯集成功

圖 7-3 驗證用幾何的尺寸與驗收數字對照。

捌、Antigravity 的獨立複驗

本章的內容不是我寫的。Antigravity 取得自己的 Onshape MCP 連線後，在沒有看過本報告任何內容的前提下，被交付同一個幾何規格，自行完成建模與量測。以下是它的實際遭遇。

一、它得到的數字

獨立量測結果

204712.38898038468 mm³

與通道 A 完全相同。而它的建構方式與我不同——我用「草圖畫圓再 opExtrude」，它直接用 fCylinder 造圓柱。兩條不同的路走到同一個數字，這才是有意義的複驗。

二、它踩到的坑（與我不同）

它遇到的錯誤	它如何定位
以為基元函式叫 opCuboid / opCylinder，實際報 Function opCuboid with 3 argument(s) not found	用 search_featurescript_documentation 查 primitives，確認正確前綴是 f 而非 op
在 lambda 中寫 id + "base"，報 Can not add map and string	發現 test_featurescript 傳入的 id 實際上是 map，不是 Id 型別
改傳陣列後報 No matching function for qCreatedBy(array, EntityType)	確認 Id 不能以裸陣列替代，改用 test_feature 取得真正的 Id
opBoolean 失敗	回頭查它先前印出的官方文件片段，找到明載的一句：Target bodies are not used by UNION

表 8-1 Antigravity 獨立作業時遭遇的問題

它找到了我沒找到的證據

同一個 opBoolean 的坑，我是靠實驗推翻假設才發現的（先誤判為「兩體僅面接觸」，讓柱體下沉 1mm 仍失敗才轉向）。它則直接在官方文件裡找到白紙黑字的一句：

Target bodies are not used by UNION.

這正是獨立複驗的價值——不是確認我對，而是可能給出更好的證據。

三、它自陳學到的事實

- FeatureScript 中布林等「操作」以 op 為前綴，但最基礎的幾何基元創建函式以 f 為前綴（fCuboid、fCylinder）
- opBoolean 的 UNION 完全不接受 targets 參數，這與 CAD 介面裡「選目標零件與工具零件」的思維不同
- 可以跑一個最簡單的 lambda，從回傳值的 libraryVersion 欄位取得當前版本號，避免寫錯版本標頭
- evVolume 回傳的是 ValueWithUnits 而非純數字，要除以 (millimeter ^ 3) 才會得到乾淨的 mm³ 數值

與我的結論的交集與差異

交集：opBoolean 的 UNION 不能給 targets、版本號要動態取得——兩邊獨立得到相同結論。

差異：我遇到的 evBox3d 編譯失敗它完全沒碰到（因為它沒用那個函式）；它遇到的 f 前綴命名與 lambda 中 id 為 map 的問題我也沒碰到。**兩份清單互補，合起來才完整。**

值得一提的是，本報告第伍章記錄過一次教訓：初次請 Antigravity 診斷編譯錯誤時，它給出四條全錯的「確定錯誤」。差別在於——那一次它是在憑記憶臆測，這一次它手上有工具，每個判斷都能驗證。同一個模型，有無驗證管道的差距就是這麼大。

玖、給後來者的操作準則

以下十條是本次實驗壓縮後的結論，可直接照做。前七條關於 FeatureScript 與成本，後三條關於驗證設計——後者更重要，因為它們決定你會不會在不知情的狀況下寫出錯的東西。

一、版本標頭以實測為準

先跑一次 `test_featurescript`，看回傳的 `libraryVersion`（本次為 3070），標頭與 `import` 版本都用它。不要相信模型寫的版本號——初稿寫的是 2420，直接導致 `no features found`。

二、除錯迴圈用自建探測器，不要用 `test_feature`

寫入 Feature Studio 後取 `featurespecs`，看找到幾個特徵。每次兩呼叫，是 `test_feature` 的一半，且訊號明確。

三、不要相信空輸出

`put_featurescript` 的說明宣稱會回報錯誤，實測對壞碼也回空字串。任何工具在你依賴它之前，先餵一個確知會失敗的輸入試它會不會叫。

四、幾何驗證用「建模與量測合一」的 `lambda`

以 `test_featurescript` 在單次呼叫內建模、量測、回傳數字，一次只花一次額度。`lambda` 內沒有 `id`，用 `["名字"] as Id` 自建。

五、標準函式庫的事實去查文件或做實驗

兩次關鍵病因模型都沒說中，第一次還給了四條言之鑿鑿但全錯的診斷。定位病因請用二分法：先把本體清空確認 `precondition` 無誤，再逐段加回。

六、把不吃額度的工作全部外包

出題、寫初稿、產生假設清單、改寫重構、文件草擬——這些交給訂閱制的代理，額度不用會過期。真正稀缺的是 Onshape 的年度呼叫次數。

七、每次呼叫都要留痕

記錄工具名稱、參數、完整輸出。本次曾因為只存了前 300 字元，被迫重跑一次四呼叫的測試——同一份資訊付了兩次錢。

八、要有可打開的產物

記憶體中算完就丟的驗證等於沒有驗證。務必留下任何人都能打開的網址，並且自己先打開一次——工具回報「已建立」與「它還在」是兩件事。

九、量測要有第二條獨立通道

同一支程式量測自己產生的東西，錯了也看不出來。本案的做法是：FeatureScript 內量一次、REST 端點量一次，兩者差值再用解析解對證。

十、複驗要讓對方自己動手

把結論交給另一個模型改寫，得到的是同一個盲點的第二種寫法。要它自己連線、自己操作、自己撞牆，它的結論才有獨立性，也才可能給出比你更好的證據。

一、適用邊界

本報告的實驗對象是單一自訂特徵，規模不大。若要處理需要跨多個 Feature Studio、或涉及大量幾何反覆試誤的題目，2,500 次的年度額度會相當吃緊——建議先以本文的方法量測該類題目的實際單價，再決定是否升級方案。

另需說明：本次全部驗證都在一個專為此建立的暫存文件內進行，未觸及既有設計檔。以代理式 AI 操作 CAD 時，這是最基本的防護——不論權杖的權限範圍為何，把工作區隔離開來都是應該的。

附錄 smartBoss 完整原始碼

以下為通過全部驗收的版本 (v5)。兩處關鍵修正已在程式碼中以註解標示。

```
FeatureScript 3070;
import(path : "onshape/std/geometry.fs", version : "3070.0");

annotation { "Feature Type Name" : "Smart Boss" }
export const smartBoss = defineFeature(function(context is Context, id is Id, definition is map)
  precondition
  {
    annotation { "Name" : "Faces", "Filter" : EntityType.FACE && GeometryType.PLANE }
    definition.faces is Query;

    annotation { "Name" : "Boss Diameter" }
    isLength(definition.bossDiameter, LENGTH_BOUNDS);

    annotation { "Name" : "Boss Height" }
    isLength(definition.bossHeight, LENGTH_BOUNDS);

    annotation { "Name" : "Add Fillet" }
    definition.addFillet is boolean;

    if (definition.addFillet)
    {
      annotation { "Name" : "Fillet Radius" }
      isLength(definition.filletRadius, LENGTH_BOUNDS);
    }
  }
  {
    var faces = evaluateQuery(context, definition.faces);
    var toolBodies = [];
    var i = 0;

    for (var face in faces)
    {
      var faceId = id + unstableIdComponent(i);

      // evBox3d 在 geometry.fs 下編譯不過 ( 實測 2026-08-29 ) ,
      // 改用 evFaceTangentPlane 直接取得面上一點與外法線。
      var tp = evFaceTangentPlane(context, {
        "face" : face,
```

```

        "parameter" : vector(0.5, 0.5)
    });

    var sketchId = faceId + "sketch";
    // tp 是 Plane 值，必須用 newSketchOnPlane；newSketch 收的是平面 Query。
    var sk = newSketchOnPlane(context, sketchId, { "sketchPlane" : tp });
    skCircle(sk, "circle", {
        "center" : vector(0, 0) * millimeter,
        "radius" : definition.bossDiameter / 2
    });
    skSolve(sk);

    var extrudeId = faceId + "extrude";
    opExtrude(context, extrudeId, {
        "entities" : qSketchRegion(sketchId),
        "direction" : tp.normal,
        "endBound" : BoundingType.BLIND,
        "endDepth" : definition.bossHeight
    });

    toolBodies = append(toolBodies, qCreatedBy(extrudeId, EntityType.BODY));
    i += 1;
}

if (size(toolBodies) > 0)
{
    var booleanId = id + "boolean";
    // UNION 時「所有」待合併實體都要放進 tools；給了 targets 會得到
    // BOOLEAN_BAD_INPUT ( targets 只用於 SUBTRACTION / INTERSECTION )。實測 2026-08-29。
    opBoolean(context, booleanId, {
        "tools" : qUnion(append(toolBodies, qOwnerBody(definition.faces))),
        "operationType" : BooleanOperationType.UNION
    });

    if (definition.addFillet)
    {
        var filletEdges = qEntityFilter(qCreatedBy(booleanId), EntityType.EDGE);
        if (!isQueryEmpty(context, filletEdges))
        {
            opFillet(context, id + "fillet", {
                "entities" : filletEdges,
                "radius" : definition.filletRadius
            });
        }
    }
}
}

```

```
});
```

程式碼另存於 github.com/henrychao521/antigravity-bridge 的 `onshape-lab/rounds/v5.fs`